— AZ-104 // CONTAINERS

Container Instances & Container Apps

Two managed container services with very different jobs — ACI for simple, short-lived containers with no orchestration overhead, and Container Apps for scalable, event-driven microservices built on Kubernetes without the cluster management.

ACI vs Container Apps — pick one

Both run containers without managing VMs. The difference is scope: ACI is the simplest possible way to run a container; Container Apps is a full microservices platform built on top of Kubernetes, with autoscaling, traffic splitting, and Dapr built in.

AZURE CONTAINER INSTANCES

ACI

- → Fastest path to a running container — no cluster, no config
- → Billed per second of CPU and memory used
- → Single container or a container group (multiple containers on the same host)
- → No autoscaling — you control instance count manually
- → Best for: batch jobs, CI/CD build agents, short-lived tasks, burst overflow
- → Supports VNet injection for private networking

AZURE CONTAINER APPS

Container Apps

- → Managed Kubernetes environment — no cluster to operate
- → KEDA-based autoscale: HTTP traffic, queue depth, CPU, cron
- → Scale to zero — pay nothing when idle
- → Traffic splitting between revisions (e.g. 90/10 canary)
- → Dapr sidecar support for microservice patterns
- → Best for: APIs, microservices, event-driven workers, long-running apps

Eight terms you need cold

Container Group (ACI)

A collection of containers deployed together on the same host in ACI, sharing network namespace, lifecycle, and storage volumes — similar to a Kubernetes pod.

Tip: All containers in a group share the same public IP and port space — map different ports per container to avoid conflicts.

Container Apps Environment

The secure boundary and shared infrastructure (virtual network, logging, certificates) that one or more Container Apps live inside. Acts as the deployment target.

Tip: All apps in the same environment can communicate over a private internal network using their app name as the hostname.

Revision

An immutable snapshot of a Container App's configuration and code. Every deploy creates a new revision. Traffic can be split across revisions for canary and A/B deployments.

Tip: Old revisions can be kept active for rollback — traffic weight 0% means they're running but receiving no traffic.

KEDA Scaling

Kubernetes-based Event Driven Autoscaling — the engine behind Container Apps autoscale. Scales on HTTP requests, Azure Service Bus queue depth, Event Hubs, CPU, memory, or cron schedules.

Tip: Scale to zero is only available for event-driven triggers, not CPU/memory-based scaling.

Ingress

The HTTP configuration for a Container App — whether it accepts external (internet) or internal (environment-only) traffic, the target port, and whether HTTPS is enforced.

Tip: Set ingress to `internal` for backend services that should only be reachable by other apps in the same environment.

Dapr

Distributed Application Runtime — an optional sidecar that handles service discovery, pub/sub messaging, state management, and secrets for microservices, without code changes.

Tip: Enable Dapr with a single flag — apps call Dapr's API on localhost and Dapr handles the rest.

Container Registry (ACR)

Azure Container Registry — stores and serves Docker images. Both ACI and Container Apps pull from ACR using managed identity (no credentials needed if configured correctly).

Tip: Assign the `AcrPull` role to the container resource's managed identity — never embed registry credentials in config.

Init Container

A container that runs to completion before the main container starts — used for setup tasks like database migrations, config injection, or secret fetching.

Tip: Supported in both ACI (container groups) and Container Apps. If the init container fails, the main container does not start.

FEATURE	ACI	CONTAINER APPS
Autoscale	No — manual	**Yes — KEDA (HTTP, queue, CPU, cron)**
Scale to zero	No	**Yes — event-driven triggers**
Traffic splitting	No	**Yes — per revision**
Dapr support	No	**Yes — built in**
Billing model	**Per second (CPU + memory)**	**Per vCPU-second + requests**
VNet integration	**Yes — VNet injection**	**Yes — environment-level**
Best for	Batch, short tasks, burst	APIs, microservices, event workers

EXAM GOTCHA — ACI VS AKS VS CONTAINER APPS

Three container services, three different use cases: **ACI** = simplest single container, no orchestration. **Container Apps** = managed microservices platform, no cluster to manage. **AKS (Azure Kubernetes Service)** = full Kubernetes cluster you control — use when you need cluster-level config, custom operators, or direct Kubernetes API access that Container Apps doesn't expose.

Choosing and deploying a container service

Run through this every time you need to run a container workload in Azure.

START

You need to run a containerized workload in Azure

DECIDE

Do you need full Kubernetes cluster control (custom operators, node pools, cluster networking)?

YES

AKS — full managed Kubernetes cluster (outside this guide's scope)

NO

Continue — ACI or Container Apps

DECIDE

Is this a short-lived task, batch job, or CI/CD agent?

YES — SHORT-LIVED

ACI — pay per second, zero setup, dispose when done

NO — LONG-RUNNING / SCALABLE

Container Apps — continue below

DECIDE

What drives scaling for this app?

HTTP TRAFFIC

Container Apps with HTTP scaler — scales on concurrent requests

QUEUE / EVENTS

Container Apps with KEDA event trigger (Service Bus, Event Hubs, Storage Queue)

SCHEDULE

Container Apps with cron scaler


DECIDE
Should this app be reachable from the internet?
YES — PUBLIC API
Set ingress to External — Container Apps assigns a public FQDN
NO — INTERNAL ONLY
Set ingress to Internal — only reachable inside the environment
NO HTTP AT ALL
Disable ingress — background workers, queue consumers
DECIDE
Where is the container image stored?
ACR
Enable managed identity on the app → assign AcrPull role — no stored credentials
DOCKER HUB / OTHER
Configure registry credentials in the app — rotate regularly
DONE
Container running with the right service, scale trigger, ingress, and registry auth

Tools for ACI and Container Apps

Azure CLI — ACI

BEST FOR: fast container deploys, batch jobs

Stand up a container in seconds — no YAML, no cluster. The fastest path from an image to a running container in Azure.

```
az container create \
  -g rg-prod -n mycontainer \
  --image
mcr.microsoft.com/azuredocs/aci-
helloworld \
  --cpu 1 --memory 1.5 \
  --ports 80 \
  --dns-name-label myapp-$(date
+%s)

az container show \
  -g rg-prod -n mycontainer \
  --query instanceView.state

az container logs \
  -g rg-prod -n mycontainer
```

Azure CLI — Container Apps

BEST FOR: deploy apps, manage revisions, set scaling

Create environments, deploy apps, update images, and configure scale rules — all from one CLI extension.

```
az containerapp env create \
  -g rg-prod -n env-prod \
  --location eastus

az containerapp create \
  -g rg-prod -n myapi \
  --environment env-prod \
  --image myacr.azurecr.io/myapi:v1
\
  --target-port 8080 \
  --ingress external \
  --min-replicas 1 --max-replicas
10

az containerapp update \
  -g rg-prod -n myapi \
  --image myacr.azurecr.io/myapi:v2
```

⟨⟩ Bicep — Container Apps

BEST FOR: IaC — environment + apps as code

Deploy environment and apps together in a template — version-controlled scaling rules and ingress configuration.

```
resource app
'Microsoft.App/containerApps@2023-
05-01' = {
  name: 'myapi'
  location: location
  properties: {
    managedEnvironmentId: env.id
    configuration: {
      ingress: { external: true,
targetPort: 8080 }
    }
    template: {
      containers: [{
        name: 'myapi'
        image:
'myacr.azurecr.io/myapi:v1'
        resources: { cpu: '0.5',
memory: '1Gi' }
      }]
      scale: { minReplicas: 1,
maxReplicas: 10 }
    }
  }
}
```

⟨⟩ Bicep — ACI Container Group

BEST FOR: IaC batch container deployments

Define container groups as code — reproducible batch jobs or sidecar patterns deployed consistently every time.

```
resource cg
'Microsoft.ContainerInstance/
containerGroups@2023-05-01' = {
  name: 'myjob'
  location: location
  properties: {
    osType: 'Linux'
    restartPolicy: 'Never'
    containers: [{
      name: 'worker'
      properties: {
        image:
'myacr.azurecr.io/worker:latest'
        resources: {
          requests: { cpu: 1,
memoryInGB: 1 }
        }
      }
    }]
  }
}
```

ACR Build & Push

BEST FOR: building images in Azure without a local Docker

Build container images directly in ACR from source — no local Docker daemon needed. Essential for cloud-only CI pipelines.

```
az acr build \
  --registry myacr \
  --image myapi:v1 .

az acr run \
  --registry myacr \
  --cmd "myacr.azurecr.io/myapi:v1"
/dev/null
```

SERVICE	STARTUP TIME	AUTOSCALE	SCALE TO ZERO	BILLING
ACI	**Seconds**	No	No	Per second
Container Apps	**Seconds–minutes**	**Yes — KEDA**	**Yes**	Per vCPU-s + requests
AKS	Minutes (cluster)	**Yes — HPA/KEDA**	**Yes (with KEDA)**	Per node VM

Principles worth internalizing

CONTAINER APPS REVISION MODES

Container Apps has two revision modes: **Single** — only one active revision at a time, new deploy replaces the old. **Multiple** — multiple revisions run simultaneously with configurable traffic weights. Use Multiple for canary deployments (e.g. 90% traffic to v1, 10% to v2), then shift to 100% once validated.

EXAM GOTCHA — ACI RESTART POLICY

ACI has three restart policies: **Always** (default — restarts on failure), **OnFailure** (restarts only if exit code is non-zero), and **Never** (runs once and stops). For batch jobs that run to completion, set `--restart-policy Never` — otherwise ACI will loop-restart the completed container indefinitely, running up costs.

Six mistakes that show up on the exam

Using ACI when autoscaling is needed

ACI has no autoscale — you manage instance count manually. For workloads that need to scale with traffic or queue depth, use Container Apps.

Leaving ACI restart policy as Always for batch jobs

A batch job that exits with code 0 will be immediately restarted with the default Always policy — set it to Never or OnFailure for one-shot jobs.

Setting Container Apps ingress to external for a backend service

Internal backend services (databases, workers, internal APIs) should use Internal ingress — they're reachable by name inside the environment with no public exposure.

Embedding ACR credentials in container config

Assign the AcrPull RBAC role to the container resource's managed identity — no credentials to rotate or accidentally commit to source control.

Expecting scale-to-zero with CPU scaling

CPU/memory-based scaling in Container Apps has a minimum of 1 replica — only event-driven triggers (HTTP, queue, cron) support true scale to zero.

Confusing Container Apps with AKS

Container Apps is built on Kubernetes but hides the cluster entirely. AKS gives you the full Kubernetes API and cluster-level control. Use AKS only when Container Apps can't satisfy a specific Kubernetes requirement.
